ONE-PAGE WORKFLOW · ULX5M MODULES ON A RASPBERRY PI CM5 IO BOARD

From KiCad boards to an animated render

How an AI agent team turned three printed-circuit-board designs into a 60-second, 1920×1080 Cycles animation — every step scripted, every step checked.

KiCad 10 pcbnew Python · kicad-cli → **pcb2blender** pcb3d_importer 2.20.0 → **Blender 5.1.2** Cycles · Workbench · headless → **Flamenco 3.9.3** distributed frames → **ffmpeg** x264 + AAC → MP4

How AI works in Blender

The agent never clicks in the Blender UI. One script — `build_scene.py` — rebuilds the whole scene from KiCad data, so every change is a code change: repeatable, diffable, committed.

- **Write** Python against Blender's API (`bpy`) — never clicks in the UI
- **Run headless** `blender -b scene.blend --python tools/<script>.py -- <args>`
- **Look** Workbench stills, storyboard, playblast, low-sample Cycles stills — the agent reads the images
- **Measure** check scripts return numbers: `label_check`, `pin1_check`, `bom_check`, `k04_check`, `glow_check`, `check_portable` + unit tests
- **Approve** storyboard sheet → producer's numbered remarks → next round; final render only on the producer's word

↻ any number ≠ 0 → fix the script, rebuild, check again

Other ways exist — e.g. Blender MCP, where a model drives a live Blender session over a socket and looks at viewport screenshots. **Not used here:** headless scripts are what make the pipeline repeatable and runnable on render workers.

THE PIPELINE IN 12 STEPS

A Board data

01 Pin the board versions
`git log · git archive`
board repos → exact commit per board (silkscreen must match)
Jelena
✓ `.kicad_pcb` byte-identical to tagged commit (sha256); QA repeats it

02 Extract board data
`extract_board.py` (pcbnew Python, no GUI) · `bom_extract.py`
`.kicad_pcb` → JSON: pads, footprints, power tracks, BOM
Jelena
✓ one JSON feeds every later step

03 Export 3D models
`pcb_render.sh` → `export_pcb3d.py` + pcb2blender · `kicad-cli pcb export glb`
board → `.pcb3d` (board + parts + materials) / `.glb`
Jelena
✓ `models_audit.py` lists footprints without a model

B Geometry you can prove

04 Layer textures
`kicad-cli pcb export svg` → `make_layers.py` · `kicad_layers.py` · `make_skin.py` · `make_explode.py`
copper / mask / silk → PNG masks at 40 px/mm, skin, exploded sheets
Jelena · Grga
✓ `test_make_layers`, `test_make_explode`

05 Pin 1 ↔ pin 1 mating
`mating.py` · `pin1_check.py` · `pin1_map.py`
pads of module + carrier → rigid 2D transform, no mirroring
Jelena; Grga cross-checks
✓ residual 0.000 mm; a pin left out of the fit lands on its pad

06 Missing models
`bom_check.py` · `extra_models.py` · `vrml.py` · `pkg3d.py` · `dnp.py`
BOM vs scene → added models; DNP connectors hidden
Jelena
✓ parts without a model = 0 / 0 / 0 on all three boards

07 Labels (callouts)
`label_specs.py` (text SSOT) · `labels.py`
spec → 3D text facing camera, ≥ 4 s on screen
Jelena; producer approves text
✓ `label_check`: overlaps 0 · crossings 0 · off-frame 0

C Look & story

08 Materials, chip marks, light, camera
`build_scene.py` · `marks.py` · `make_mark_gs_*.py` · `pin1_pads.py` · `cycles_still.py`
material TOML + datasheets → laser marks by pad 1, KEY/FILL/RIM, AgX, Bloom, DOF
Jelena, from Grga's visual spec
✓ Cycles stills read by eye · `test_marks` · `glow_check`

09 Shot list & storyboard
`make_frames.py` · `stack.py` · `make_timeline.py` · `storyboard.py` · `schedule.py` · `still.py`
idea → shots K01–K09, timeline, real Workbench frames
Grga; producer approves
✓ every remark tracked: remark → state → frame · `test_schedule`

10 Playblast & sound
`playblast.py` · `playblast_all.sh` · `make_soundtrack.py` · ffmpeg
scene → 640×360 Workbench MP4; 120 BPM synth from the shot schedule
Jelena
✓ sound cues within ±3 frames of the picture

D Render & deliver

11 Estimate & portable file
`render_probe.py` · `render_estimate.py` · `make_portable.py` · `check_portable.py` · `preview_render.py`
probe one frame per shot class → hours; self-contained `.blend` without the add-on
Jelena
✓ `check_portable`: undefined nodes 0 · unpacked files 0

12 Distributed render, MP4, QA
Flamenco job `simple-blender-render` (chunk 3) · `straza.py` · `sync_share.py` · `sastavi_mp4.py`
`.blend` + soundtrack → PNG frames from several workers → MP4 with sound
Jelena render · Potjeh QA
✓ workers match (mean diff < 0.01/255); frames 0–1799, no gaps

⚠ PITFALLS WE HIT — AND THE CHECK THAT NOW CATCHES EACH

- **White boards in Cycles, no error.** pcb2blender materials are custom shader nodes; without the `pcb3d_importer` add-on Cycles renders them white (Workbench hides it). → `cycles_still.py` refuses to run without it; `make_portable.py` bakes plain nodes; `check_portable.py` = 0 undefined nodes.
- **Module 2 mm off its slot.** pcb2blender centres a board on its outline, not on KiCad coordinates → calibrate on a reference chip; `pin1_check.py` reports the offset in mm.
- **Never align by eye.** Orientation comes from the pads in `.kicad_pcb`, never from how the model looks.
- **Cutting content ≠ faster render.** Dropping the exploded layers saved nothing — those frames were cheap. Measure per shot class first.
- **Mixed Blender versions on workers.** 5.1.2 and 5.2.x gave matching frames — verified on neighbouring frames across worker boundaries before the full job.
- **A paused worker counts as failed.** Flamenco blocklists after repeated aborts; `straza.py` pauses a busy machine and clears the blocklist when it may work again.

EVERY SCRIPT, ONE LINE EACH — 61 entries

BOARD DATA & MATING
- `extract_board.py` pads, footprints, power tracks → JSON
- `bom_extract.py` BOM from the board
- `mating.py` pin 1 → pin 1 rigid transform
- `pin1_check.py` pin 1 offset in the final frame
- `pin1_pads.py` pad 1 of chosen footprints (orients marks)
- `pin1_map.py` top view of modules on carrier
- `refs.py` footprint refs for the independent check

3D EXPORT · PCB_RENDER
- `pcb_render.sh` git rev → .pcb3d → Cycles PNG → collage
- `export_pcb3d.py` KiCad 10 → pcb2blender format
- `scene_render.py` studio scene and views
- `render_lib.py` pure logic: config, light, naming
- `compose.py` JPG on background, collage
- `pkg3d.py` parametric packages (STEP + WRL)
- `models_audit.py` footprints without a 3D model
- `make_markings.py` chip-marking masks
- `lcsc2kicad.sh` fetch a model by LCSC part number

LAYERS & LOOK
- `make_layers.py` layer SVG → PNG masks
- `kicad_layers.py` same, without KiCad
- `make_skin.py` board skin for Workbench
- `make_explode.py` exploded-layer sheets
- `extra_models.py` adds models the export missed
- `vrml.py` KiCad .wrl reader (Blender 5: no VRML)
- `dnp.py` hides unpopulated connectors
- `make_mark_gs_sdram.py` SDRAM laser mark
- `make_mark_gs_ksz.py` Ethernet PHY laser mark
- `marks.py` orients a mark by pad 1

SCENE & TIMING
- `build_scene.py` builds the whole animation from data
- `schedule.py` shot schedule (pure function)
- `label_specs.py` label texts
- `labels.py` label layout + leader lines
- `make_soundtrack.py` soundtrack from the schedule
- `patch_scene.py` applies fixes to a .blend

LOOK & CHECK
- `still.py` quick Workbench still
- `still_gs.py` same, GS board
- `cycles_still.py` low-sample Cycles still
- `storyboard.py` one real frame per shot
- `playblast.py` Workbench preview
- `playblast_all.sh` all shots in parallel + MP4
- `label_check.py` overlaps · crossings · off-frame
- `bom_check.py` parts without a model
- `k04_check.py` board leaves frame on time
- `glow_check.py` emission / glow
- `test_*.py` 38 unit tests

RENDER
- `render_probe.py` times one frame per mode
- `render_estimate.py` total time per shot class
- `preview_render.py` low-res Cycles preview
- `preview_all.sh` preview of all shots
- `make_portable.py` add-on-free render file
- `check_portable.py` its check (--factory-startup)
- `ctl.sh` start / stop a render worker
- `straza.py` worker guard: pause, wake, blocklist
- `sync_share.py` mirrors the shared render folder
- `sastavi_mp4.py` checks frames → ffmpeg MP4 + sound

STORYBOARD · GRGA
- `make_frames.py` style thumbnails
- `stack.py` 2.5D layer-stack sketch
- `make_timeline.py` timeline graphic
- `render_fp_logo.py` footprint logo → PNG
- `svg2png.py` SVG raster via headless Chromium

QA · POTJEH
- `inspect_blend.py` independent scene inspection
- `qa_scene2.py` scene QA, JSON evidence
- `qa_m2.py` M2 board QA, JSON evidence

WHO DOES WHAT — AGENTS & SKILLS

Jelena ENGINEER
`Development · TDD · DiagnosingBugs · CreateCLI`
built the pipeline and the scene

Grga DESIGNER
`FrontendDesign · Browser · Excalidraw`
shot list, visual spec, storyboards

Potjeh QA
`independent check scripts`
versions, pin 1, models, labels, sound, logos

Producer HUMAN
`approval`
signs off every round and the final render

EXTERNAL TOOLS KiCad 10 (kicad-cli, pcbnew Python) · pcb2blender · Blender 5.1.2 + Cycles / OIDN · Flamenco 3.9.3 · ffmpeg · ImageMagick · Playwright / Chromium · Python 3 (Pillow, NumPy) · git